

Assignment:-4

1. Create a class named 'Rectangle' with two data members- length and breadth and a function to calculate the area which is 'length*breadth'. The class has three constructors which are :

(a) having no parameter - values of both length and breadth are assigned zero.

(b) having two numbers as parameters - the two numbers are assigned as length and breadth respectively.

(c) having one number as parameter - both length and breadth are assigned that number.

Now, create objects of the 'Rectangle' class having none, one and two parameters and print their areas.

Code:-

```
ques1.cpp
1 //Assignment- 4
2 //Question -1
3 #include <iostream>
4 using namespace std;
5 class Rectangle {
6     int length, breadth;
7 public:
8     // (a) No parameter
9     Rectangle() {
10         length = 0;
11         breadth = 0;
12     }
13     // (b) Two parameters
14     Rectangle(int l, int b) {
15         length = l;
16         breadth = b;
17     }
18     // (c) One parameter
19     Rectangle(int x) {
20         length = breadth = x;
21     }
22     int area() {
23         return length * breadth;
24     }
25 };
26 int main() {
27     Rectangle r1;           // no parameter
28     Rectangle r2(5);        // one parameter
29     Rectangle r3(4, 6);     // two parameters
30     cout << "Area r1: " << r1.area() << endl;
31     cout << "Area r2: " << r2.area() << endl;
32     cout << "Area r3: " << r3.area() << endl;
33 }
```

Output:-

```

C:\Users\asus\Desktop\oops\ x + v
Area r1: 0
Area r2: 25
Area r3: 24

-----
Process exited after 0.2959 seconds with return value 0
Press any key to continue . . .

```

2. Redefine the above program by creating an array of objects of the class Rectangle and calculate area for each object calling different constructors. Also implement constructors with default arguments and destructor in this program.

Code:-

```

ques1.cpp ques2.cpp
1 //Assignment- 4
2 //Question -4
3 #include <iostream>
4 using namespace std;
5 class Rectangle {
6     int length, breadth;
7 public:
8     // Constructor with default arguments
9     Rectangle(int l = 0, int b = 0) {
10         length = l;
11         breadth = b;
12     }
13     int area() {
14         return length * breadth;
15     }
16     ~Rectangle() {
17         cout << "Destructor called\n";
18     }
19 };
20 int main() {
21     Rectangle r[3] = {
22         Rectangle(),           // 0,0
23         Rectangle(5),         // 5,5
24         Rectangle(4,6)
25     };
26     for(int i=0;i<3;i++) {
27         cout << "Area: " << r[i].area() << endl;
28     }
29 }

```

Output:-

```

C:\Users\asus\Desktop\oops\ × + ▾
Area: 0
Area: 0
Area: 24
Destructor called
Destructor called
Destructor called

-----
Process exited after 0.2255 seconds with return value 0
Press any key to continue . . .

```

3. Verify the following about *destructor* by writing the program:

- (i) Name should begin with tilde sign(~) and must match class name.
- (ii) There cannot be more than one destructor in a class.
- (iii) Destructors do not allow any parameter.
- (iv) They do not have any return type, just like constructors.

Code:-

```

ques1.cpp  ques2.cpp  [*] Untitled32
1  //Assignment- 4
2  //Question -3
3  #include <iostream>
4  using namespace std;
5  class Demo {
6  public:
7      Demo() {
8          cout << "Constructor called\n";
9      }
10     ~Demo() { // destructor
11         cout << "Destructor called\n";
12     }
13 };
14 int main() {
15     Demo d1;
16     {
17         Demo d2; |
18     }
19 }

```

Output:-

```

C:\Users\asus\Desktop\oops\  x  +  v
Constructor called
Constructor called
Destructor called
Destructor called

-----
Process exited after 0.247 seconds with return value 0
Press any key to continue . . .

```

4. Implement dynamic memory allocation. Use *new* and *delete* keywords. (For integer variable, float variable, integer array, float array, class objects, Array of Objects)

Code:-

```

ques1.cpp  ques2.cpp  ques3.cpp  ques4.cpp
1  //Assignment- 4
2  //Question -4
3  #include <iostream>
4  using namespace std;
5  class Test {
6  public:
7      int x;
8      void show() {
9          cout << "Value: " << x << endl;
10     }
11 };
12 int main() {
13     // 1. Integer
14     int *p = new int;
15     *p = 10;
16     cout << "Integer: " << *p << endl;
17     delete p;
18     // 2. Float
19     float *f = new float;
20     *f = 5.5;
21     cout << "Float: " << *f << endl;
22     delete f;
23     // 3. Integer Array
24     int *arr = new int[3];
25     arr[0]=1; arr[1]=2; arr[2]=3;
26     for(int i=0;i<3;i++)
27         cout << arr[i] << " ";
28     cout << endl;
29     delete[] arr;
30     // 4. Float Array
31     float *fa = new float[2];
32     fa[0]=1.1; fa[1]=2.2;
33     for(int i=0;i<2;i++)
34         cout << fa[i] << " ";
35     cout << endl;
36     delete[] fa;
37     // 5. Single Object
38     Test *t = new Test;

```

```
37 // 5. Single Object
38 Test *t = new Test;
39 t->x = 100;
40 t->show();
41 delete t;
42 // 6. Array of Objects
43 Test *obj = new Test[2];
44 obj[0].x = 10;
45 obj[1].x = 20;
46 obj[0].show();
47 obj[1].show();
48 delete[] obj;
49 }
```

Output:-

```
C:\Users\asus\Desktop\oops\  ×  +  ∨
Integer: 10
Float: 5.5
1 2 3
1.1 2.2
Value: 100
Value: 10
Value: 20

-----
Process exited after 0.2983 seconds with return value 0
Press any key to continue . . .
```